

SYNAPSE

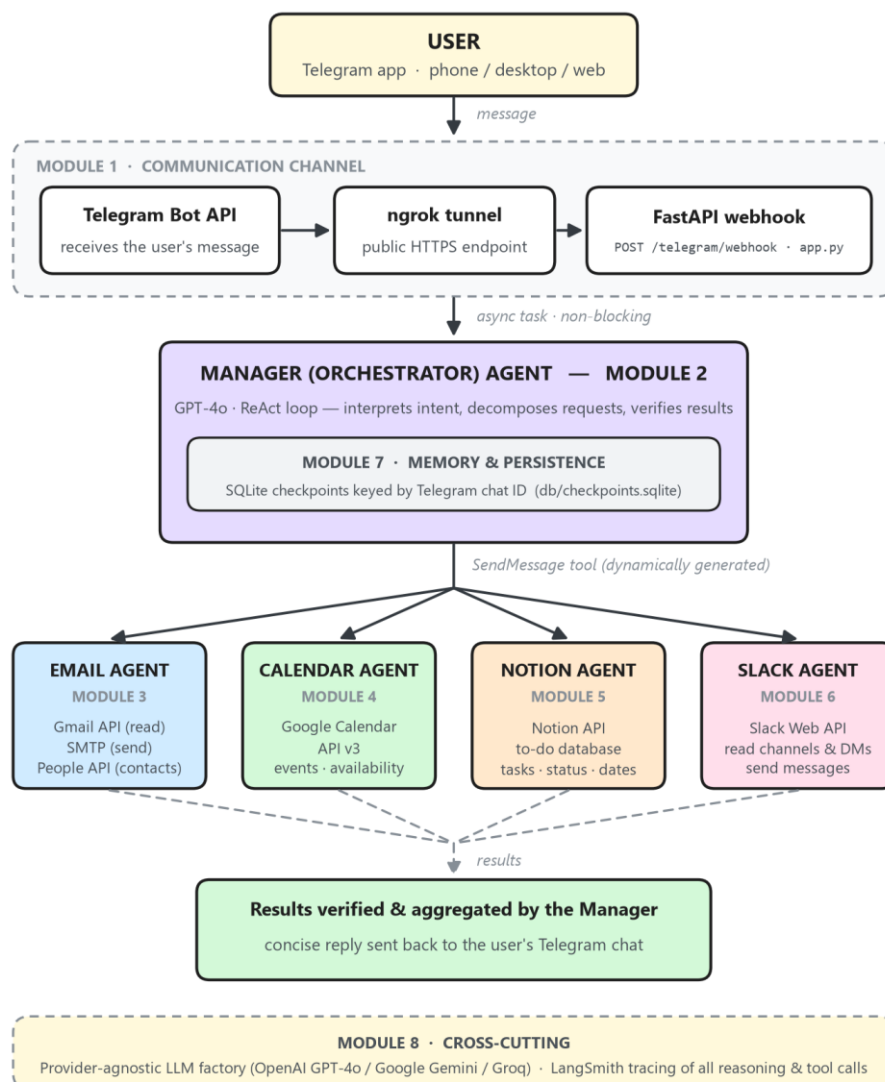
Multi-Agent Personal Assistant

System Architecture & Database Schema

1. System Architecture Overview

Synapse is a hierarchical multi-agent personal assistant built on the LangGraph agentic framework, following a supervisor–worker architecture. A Manager (orchestrator) agent receives user messages through a Telegram Bot API–integrated FastAPI webhook, interprets the intent, decomposes compound requests into discrete sub-tasks, and delegates each to one of four specialised sub-agents (Email, Calendar, Notion, Slack). Each sub-agent executes an independent ReAct reasoning loop over its own tool set and reports results back to the Manager, which verifies, aggregates, and returns a concise reply to the user on Telegram.

1.1 Architecture Diagram



1.2 Modules

#	Module	Responsibility
1	Communication Channel (Telegram / Bot API)	Receives incoming Telegram messages through a FastAPI webhook (exposed via ngrok during development) and returns replies asynchronously.
2	Manager (Orchestrator) Agent	Supervisor of the system; interprets intent, decomposes compound requests, delegates via a dynamically generated SendMessage tool, verifies outputs, composes the final reply.
3	Email Agent (Gmail)	Reads and summarises inbox mail (Gmail API), sends email via SMTP, resolves contact names to addresses using the Google People API.
4	Calendar Agent (Google Calendar)	Lists scheduled and upcoming events, checks availability, creates new events.
5	Notion Agent (Task Management)	Retrieves the to-do list from a Notion database and adds new tasks with status and due-date properties.
6	Slack Agent (Team Messaging)	Reads messages from Slack channels or DMs and sends messages on the user's behalf.
7	Memory & Persistence	Persists conversation state as SQLite checkpoints keyed by the user's Telegram chat ID — isolated, persistent multi-turn context per user.
8	LLM Provider & Observability	Provider-agnostic factory instantiating the configured LLM (OpenAI GPT-4o, Google Gemini, Groq) per agent; LangSmith tracing of all reasoning and tool calls.

1.3 End-to-End Message Workflow

1. **Message arrives.** The user texts the bot; Telegram delivers a JSON update to the registered webhook URL (ngrok tunnel → POST /telegram/webhook in app.py). The handler acknowledges immediately and processes the message in an asynchronous background task, keeping the webhook non-blocking under concurrent load.
2. **Memory context loads.** The Telegram chat ID becomes the LangGraph thread_id. The SqliteSaver checkpointer restores the user's full prior conversation from db/checkpoints.sqlite before the Manager reasons.
3. **Manager interprets and decomposes.** The Manager agent (ReAct loop, GPT-4o by default) analyses the request and splits compound asks into discrete sub-tasks.
4. **Delegation via SendMessage.** The orchestrator generates the SendMessage tool at startup, embedding each sub-agent's name and description into the tool schema; the Manager selects a recipient and forwards the sub-task as a fresh instruction.
5. **Sub-agents execute ReAct loops.** Each worker reasons over its own tools only (Gmail/SMTP/People, Calendar, Notion, Slack), observing tool results and iterating until the sub-task completes.

6. **Verification and aggregation.** Sub-agent results return as tool outputs inside the Manager's loop; the Manager checks them against the request, re-delegates on failure, and composes one concise summary.
7. **Reply and checkpoint.** The reply is sent to the originating chat via the Telegram Bot API (Markdown with plain-text fallback), and the whole exchange is checkpointed to SQLite for future turns.

2. Database Design

Synapse keeps its local persistence deliberately minimal: a single SQLite file, `db/checkpoints.sqlite`, containing two tables that are created and managed automatically by LangGraph's `SqliteSaver`. All business data (emails, events, tasks, team messages) remains in the external services — the local database stores only conversation memory.

2.1 Table: checkpoints — conversation state snapshots

Column	Type	Constraints	Description
<code>thread_id</code>	TEXT	NOT NULL, PK(1)	The user's Telegram chat ID — the multi-user isolation key.
<code>checkpoint_ns</code>	TEXT	NOT NULL, PK(2), default ""	Namespace for subgraphs (empty string for the main graph).
<code>checkpoint_id</code>	TEXT	NOT NULL, PK(3)	Unique, time-ordered identifier of this snapshot.
<code>parent_checkpoint_id</code>	TEXT	NULL	ID of the previous snapshot; forms a linked chain of states.
<code>type</code>	TEXT	NULL	Serialization format of the checkpoint blob.
<code>checkpoint</code>	BLOB	NULL	Serialized conversation state: full message history, tool calls, and results at this point in time.
<code>metadata</code>	BLOB	NULL	Bookkeeping: producing step, writes performed, etc.

Primary key: (`thread_id`, `checkpoint_ns`, `checkpoint_id`)

2.2 Table: writes — pending state updates

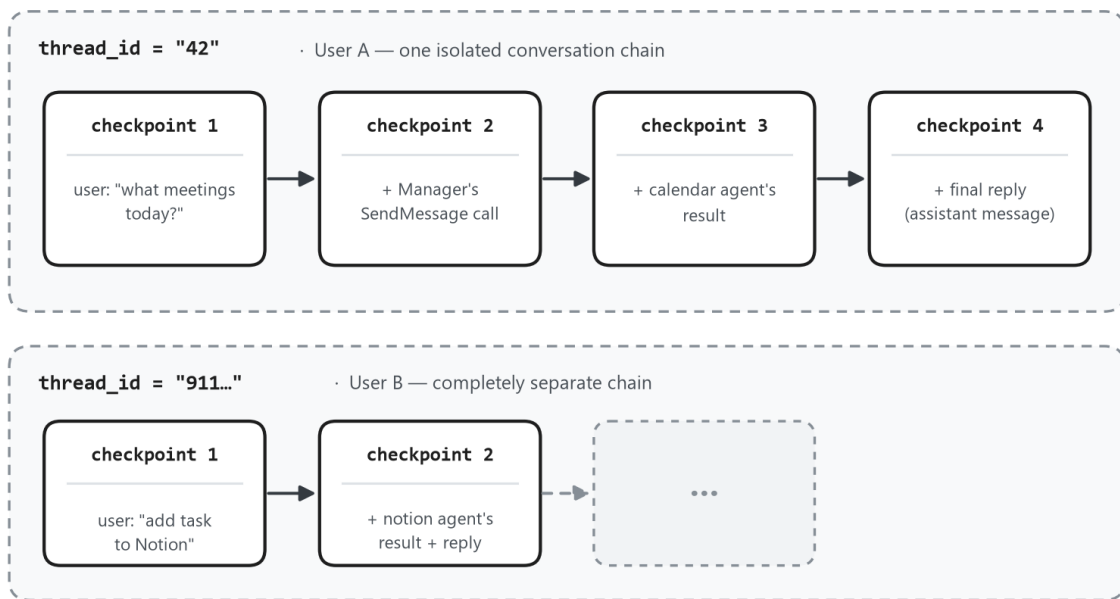
Column	Type	Constraints	Description
<code>thread_id</code>	TEXT	NOT NULL, PK(1)	Same meaning as in checkpoints (Telegram chat ID).
<code>checkpoint_ns</code>	TEXT	NOT NULL, PK(2), default ""	Subgraph namespace.
<code>checkpoint_id</code>	TEXT	NOT NULL, PK(3)	Checkpoint this write belongs to.
<code>task_id</code>	TEXT	NOT NULL, PK(4)	Graph task (node execution) that produced the write.
<code>idx</code>	INTEGER	NOT NULL, PK(5)	Order of the write within the task.
<code>channel</code>	TEXT	NOT NULL	LangGraph state channel being updated (e.g. messages).
<code>type</code>	TEXT	NULL	Serialization format of the value.
<code>value</code>	BLOB	NULL	The serialized state update itself.

Primary key: (`thread_id`, `checkpoint_ns`, `checkpoint_id`, `task_id`, `idx`)

The writes table is LangGraph's write-ahead staging area: updates produced by a graph node are recorded here before being folded into the next checkpoint, making agent execution resumable after a crash.

2.3 How conversations map to rows

Every dialogue turn appends new checkpoints to the chain for that user's thread. Each user (Telegram chat ID) owns a fully independent chain:



One row per snapshot in the checkpoints table — PK (thread_id, checkpoint_ns, checkpoint_id).
Each incoming message loads the thread's latest checkpoint, runs the turn, and appends new snapshots linked by parent_checkpoint_id.

On each incoming message, LangGraph loads the latest checkpoint for the thread, replays it as the Manager's memory, runs the turn, and appends new checkpoints. Only the Manager agent has the checker; the four worker agents are deliberately stateless and receive everything they need in the Manager's instruction.

2.4 External Data Stores

The full data landscape consists of the local SQLite store plus five external, service-owned stores accessed exclusively through their official APIs:

Store	Owner	Data	Access Path
checkpoints.sqlite (2 tables)	Synapse (local)	Per-user conversation memory	LangGraph SqliteSaver
Gmail mailbox	Google	Emails	Gmail API (read) + SMTP over SSL (send)
Google Calendar	Google	Events	Calendar API v3
Google Contacts	Google	Name → email mapping	People API
Notion database	Notion	To-do tasks	Notion API
Slack workspace	Slack	Team messages	Slack Web API

2.5 Notion To-Do Database Schema (user-designed)

The one external store designed by the team is the Notion to-do database. The code expects exactly these properties, with these exact names:

Property	Notion Type	Purpose / Allowed Values
Title	Title	The task text.
Status	Status	Task state — options: "Not started", "In progress", "Done". New tasks default to "Not started".
Date	Date	Due date; used for date-filtered queries when retrieving the to-do list.

2.6 Design Rationale

- **Minimal local state:** external services remain the single source of truth; Synapse stores no copies of emails, events, or tasks. This avoids synchronisation problems and keeps personal data out of the local file.
- **Checkpoints instead of a plain messages table:** checkpointing the whole graph state (not just chat text) lets a multi-step agent run resume after interruption and provides per-user memory with zero custom persistence code.
- **Chat-ID-keyed threads:** using the Telegram chat ID as `thread_id` gives every user an isolated, persistent multi-turn context automatically — no user table or session management required.
- **SQLite as the engine:** zero-configuration, file-based, and sufficient for a single-server assistant; the checkpointer interface allows swapping to Postgres for scale without changing agent code.